

SQL Query Document

All Queries Used in Phase 2 Analysis

Team: Event Horizon

Competition: Data Vortex 2026

Phase 2: SQL Analysis & Insight Generation

1. Schema & View Definition

Database Schema (schema.sql)

Relational schema loaded into SQLite with strict constraints, foreign keys, and indexes.

```
PRAGMA foreign_keys = ON;

CREATE TABLE users (
  user_id          TEXT PRIMARY KEY NOT NULL,
  city             TEXT,
  country          TEXT,
  language         TEXT,
  account_created  TEXT,
  follower_count   REAL CHECK (follower_count IS NULL OR follower_count >= 0),
  location_was_null INTEGER,
  language_was_null INTEGER,
  account_created_was_null INTEGER
);

CREATE TABLE posts (
  post_id          TEXT PRIMARY KEY NOT NULL,
  user_id          TEXT NOT NULL,
  platform         TEXT,
  text_content     TEXT,
  timestamp        TEXT,
  likes            REAL,
  shares           INTEGER CHECK(shares >= 0),
  comments         INTEGER CHECK(comments >= 0),
  platform_was_null INTEGER,
  FOREIGN KEY (user_id) REFERENCES users (user_id)
);

CREATE INDEX idx_posts_user ON posts(user_id);
CREATE INDEX idx_posts_platform ON posts(platform);
```

Engagement View (v_post_engagement)

Centralizes engagement calculation. No COALESCE used - NULL propagation is intentional.

```
CREATE VIEW v_post_engagement AS
SELECT
  p.*,
  (p.likes + p.shares + p.comments) AS total_engagement,
  CASE WHEN p.likes IS NULL OR p.shares IS NULL OR p.comments IS NULL
    THEN 1 ELSE 0 END              AS has_null_component
FROM posts p;
```

2. E3 - Average Engagement by Platform (Easy)

Goal: Calculate average likes, shares, and comments for each platform. Identify the platform with the highest average total engagement.

```
SELECT
    platform,
    COUNT(post_id) AS post_count,
    ROUND(AVG(likes), 2) AS avg_likes,
    ROUND(AVG(shares), 2) AS avg_shares,
    ROUND(AVG(comments), 2) AS avg_comments,
    ROUND(AVG(total_engagement), 2) AS avg_total_engagement
FROM v_post_engagement
WHERE platform IS NOT NULL
    AND likes >= 0
GROUP BY platform
ORDER BY avg_total_engagement DESC;
```

3. M1 - Location Engagement Ranking (Medium)

Goal: Calculate total engagement by users from each location. Rank locations from highest to lowest.

```
SELECT
    u.city,
    u.country,
    COUNT(p.post_id) AS post_count,
    SUM(p.total_engagement) AS total_engagement,
    ROUND(AVG(p.total_engagement), 2) AS avg_engagement_per_post
FROM users u
JOIN v_post_engagement p ON u.user_id = p.user_id
WHERE p.likes >= 0
GROUP BY u.city, u.country
ORDER BY total_engagement DESC
LIMIT 10;
```

4. H3 - Platform Performance vs Own Average (Hard)

Part 1: Diagnostic - Is the 2x Threshold Reachable?

```
SELECT
  platform,
  COUNT(*)                                AS n_posts,
  ROUND(AVG(likes + shares + comments), 1) AS avg_engagement,
  ROUND(2 * AVG(likes + shares + comments), 1) AS threshold_2x,
  MAX(likes + shares + comments)           AS max_engagement,
  ROUND(CAST(MAX(likes + shares + comments) AS REAL)
        / AVG(likes + shares + comments), 3) AS max_to_mean_ratio
FROM posts
WHERE platform IS NOT NULL
  AND likes >= 0
GROUP BY platform
ORDER BY avg_engagement DESC;
```

Part 2: Literal Answer (Returns 0 Rows)

```
WITH post_engagement AS (
  SELECT
    post_id,
    platform,
    likes + shares + comments AS total_eng,
    AVG(likes + shares + comments) OVER (PARTITION BY platform)
      AS platform_avg_eng
  FROM posts
  WHERE platform IS NOT NULL
    AND likes >= 0
)
SELECT post_id, platform, total_eng,
       ROUND(platform_avg_eng, 2) AS platform_avg_eng
FROM post_engagement
WHERE total_eng >= 2 * platform_avg_eng
ORDER BY platform, total_eng DESC;
```

Part 3: Relative Outperformers (Top 1% by Platform)

```
WITH post_engagement AS (
  SELECT
    post_id,
    platform,
    likes + shares + comments AS total_eng,
    AVG(likes + shares + comments) OVER (PARTITION BY platform)
      AS platform_avg_eng,
    PERCENT_RANK() OVER (
      PARTITION BY platform
      ORDER BY likes + shares + comments
    ) AS pct_rank_in_platform
  FROM posts
  WHERE platform IS NOT NULL
    AND likes >= 0
)
SELECT
  post_id,
  platform,
  total_eng,
```

```
ROUND(platform_avg_eng, 2)           AS platform_avg_eng,  
ROUND(total_eng / platform_avg_eng, 3) AS ratio_to_platform_avg,  
ROUND(pct_rank_in_platform * 100, 2)  AS percentile_in_platform  
FROM post_engagement  
WHERE pct_rank_in_platform >= 0.99  
ORDER BY platform, ratio_to_platform_avg DESC;
```

5. Bonus 1 - Trend Detection (MoM Growth)

Goal: Calculate monthly engagement trend per platform using LAG() window function.

```
WITH monthly_stats AS (
  SELECT
    platform,
    strftime('%Y-%m', timestamp) AS month,
    SUM(total_engagement) AS monthly_engagement
  FROM v_post_engagement
  WHERE platform IS NOT NULL AND likes >= 0
  GROUP BY platform, month
)
SELECT
  platform,
  month,
  monthly_engagement,
  LAG(monthly_engagement) OVER (PARTITION BY platform ORDER BY month)
    AS prev_month_engagement,
  ROUND(
    (monthly_engagement - LAG(monthly_engagement)
     OVER (PARTITION BY platform ORDER BY month)) * 100.0 /
    NULLIF(LAG(monthly_engagement)
     OVER (PARTITION BY platform ORDER BY month), 0), 2
  ) AS mom_growth_pct
FROM monthly_stats
ORDER BY platform, month;
```

6. Bonus 2 - Anomaly Discovery (H5)

Goal: Identify corrupted posts using specific conditions discovered during Phase 1 audit.

```
SELECT
  post_id,
  CASE
    WHEN likes < 0 THEN 'Negative Likes (Imputation Artifact)'
    WHEN platform IS NULL THEN 'Missing Platform'
    WHEN text_content LIKE '%&%' OR text_content LIKE '%<%>%'
      THEN 'HTML/Entity Corruption'
    ELSE 'Other'
  END AS anomaly_type
FROM posts
WHERE likes < 0
  OR platform IS NULL
  OR text_content LIKE '%&%'
  OR text_content LIKE '%<%>%'
LIMIT 20;
```

7. Validation Script

Goal: Prove clean data load without corruption during import.

```
SELECT 'users_count' AS metric, COUNT(*) AS value FROM users
UNION ALL
SELECT 'posts_count' AS metric, COUNT(*) AS value FROM posts
```

```
UNION ALL
SELECT 'duplicate_users', COUNT(user_id) - COUNT(DISTINCT user_id) FROM users
UNION ALL
SELECT 'duplicate_posts', COUNT(post_id) - COUNT(DISTINCT post_id) FROM posts
UNION ALL
SELECT 'null_platforms', COUNT(*) FROM posts WHERE platform IS NULL
UNION ALL
SELECT 'negative_likes', COUNT(*) FROM posts WHERE likes < 0
UNION ALL
SELECT 'max_shares', MAX(shares) FROM posts
UNION ALL
SELECT 'min_shares', MIN(shares) FROM posts;
```